

语法分析

以下语法分析内容中，对于代码中构造函数、getter、setter 函数，以及用于 `std::unordered_set`、`std::unordered_map` 中作为键的类中哈希函数，以及其它简单函数并未给出代码以及说明。

1、主要流程

`syntax_generator.cpp` 作为主函数入口，读取 `syntax_rule.txt` 语法规则文件，输出 `syntax_parser.h`、`syntax_parser.cpp` 语法分析源码。

首先，读取规则文件，保存为文法，并计算 first 集、follow 集。

```
std::filesystem::path rule_file{"./resource/rule/syntax/syntax_rule.txt"};
```

```
Grammar grammar;
```

```
SyntaxRuleParser::parse(rule_file, grammar);
```

```
grammar.compute_first_sets();
```

```
grammar.compute_follow_sets();
```

在 `syntax_rule_parser.h` 中，定义了 `SyntaxRuleParser` 类解析语法规则，并将其保存到文法对象中。

在 `grammar.h` 中，定义了 `Grammar` 类用于记录文法数据，`Grammar` 使用 `symbol.h` 中的 `Symbol` 类记录终结符、非终结符，使用 `production.h` 中的 `Production` 类记录产生式，同时提供 first 集、follow 集的计算。

其次，根据文法，依次构建 LR1 状态图、action 表、goto 表，根据 action 表、goto 表构建抽象语法树。

```
LRParser parser(&grammar);
```

```
parser.build_states();
```

```
parser.construct_tables();
```

在 lr_parser.h 中，定义了 LRParser 类用于 LR1 分析，LRParser 使用 lr_item.h 中的 LRItem 类记录 LR1 项，使用 lr_state.h 中的 LRState 类记录 LR1 状态，LRState 是包含多个 LRItem 的集合，LR1 状态具有 id，根据状态 id，LRParser 记录各个状态的 action 表项，goto 表项，其中 action 表项记录对于指定符号进行的动作，在 action.h 中定义了 Action 类用于表示移进、规约、接受、错误四种动作，goto 表项记录对于指定符号下一步的状态。同时，LRParser 还提供 LR1 状态集合并为 LALR1 的计算，但在本实践中，并未处理 LR1 状态和 LALR1 状态的映射关系，因此，语法分析使用 LR1 分析过程。

最后，生成语法分析源码，语法分析源码将会利用 LR1 分析类中的结果，提供语法分析接口。

```
std::filesystem::path h_file{"/resource/source/syntax_parser.h"};
```

```
std::filesystem::path cpp_file{"/resource/source/syntax_parser.cpp"};
```

```
std::filesystem::create_directories(h_file.parent_path());
```

```
std::ofstream h{h_file};
```

```
std::ofstream cpp{cpp_file};
```

```
// 写入源码，详见“语法分析源码生成”
```

2、文法符号

符号类主要记录类型、名称，对于操作符，额外记录优先级、结合性以便处理冲突。

符号类记录特殊符号"epsilon"、"\$"，并保证唯一性，其它类只能获取其指针。

```
class Symbol
```

```
{
```

```

public:
    enum class Type
    {
        Terminal, NonTerminal, Epsilon, End
    };

public:
    static const Symbol& get_epsilon();
    static const Symbol& get_end();

    bool operator==(const Symbol& other) const
    {
        return name_ == other.name_;
    }

private:
    static Symbol epsilon;
    static Symbol end;

    Type type_;
    std::string name_;
    mutable bool is_operator_;
    mutable uint32_t precedence_;
    mutable Associativity associativity_;
};

```

其中，符号类型分为 Terminal 终结符、NonTerminal 非终结符，Epsilon、End 用于区分特殊符号。

其中，结合性由 associativity.h 中的枚举类定义，分为 Left 左结合、Right 右结合以及 None。

```

enum class Associativity
{
    Left, Right, None
};

```

3. 产生式

产生式主要记录id、左侧非终结符以及右侧符号集合，另外还记录precedence_symbol 用于代表该产生式的优先级、结合性。

```

class Production
{

```

```

public:
    const Symbol* get_precedence_symbol() const;

    bool operator==(const Production& other) const;

private:
    void compute_precedence_symbol();

private:
    uint32_t id_;
    const Symbol* left_;
    std::vector<const Symbol*> right_;
    const Symbol* precedence_symbol_;
};

```

其中，对于 precedence_symbol 的计算，实现方式：寻找最右侧首个操作符。

```

void Production::compute_precedence_symbol()
{
    precedence_symbol_ = nullptr;
    for (auto it = right_.rbegin(); it != right_.rend(); ++it)
    {
        if ((*it)->is_operator())
        {
            precedence_symbol_ = *it;
            break;
        }
    }
}

```

4. 文法

文法主要记录终结符、非终结符与产生式，并保证唯一性，其它类只能获取其指针，同时使用 start_symbol_记录作为开始符号的非终结符。另外，文法还记录 first 集、follow 集，first 集、follow 集由符号名称作为键、符号指针集合形成的映射表实现。

```

class Grammar
{
public:
    void compute_first_sets();
    void compute_follow_sets();
    std::unordered_set<const Symbol*> compute_first_of_sequence(const
std::vector<const Symbol*>& symbols) const;

```

```

        const      std::unordered_set<const      Symbol*>      get_first_set(const
std::string_view name) const;
        const      std::unordered_set<const      Symbol*>      get_follow_set(const
std::string_view name) const;

        void show() const;
        void show_first_follow() const;

private:
        const Symbol* start_symbol_;
        std::unordered_set<Symbol> terminals_;
        std::unordered_set<Symbol> non_terminals_;
        uint32_t current_production_id_;
        std::vector<Production> productions_;
        std::unordered_map<std::string,      std::unordered_set<const      Symbol*>>
first_set_map_;
        std::unordered_map<std::string,      std::unordered_set<const      Symbol*>>
follow_set_map_;
};

```

其中，对于 first 集的计算，实现方式：对于终结符、epsilon，first 集都是自身；对于非终结符，通过以下过程完成——遍历产生式，对于 A->epsilon 形式的产生式，将 epsilon 加入 A 的 first 集；对于 A->X1X2...Xn 形式的产生式，对于每个符号 Xk，将非 epsilon 符号加入 A 的 first 集，遍历每个符号 Xk 同时，通过变量记录是否每个 Xk 都可能为 epsilon，如果每个 Xk 都可能为 epsilon，将 epsilon 加入 A 的 first 集。由于 first 集不含重复项，对于产生式的每轮遍历，都会记录是否更新了 first 集，如果更新了 first 集，重复遍历产生式进行下一轮计算，否则，first 集计算完成。

```

void Grammar::compute_first_sets()
{
    first_set_map_.clear();

    for (const Symbol& terminal : terminals_)
        first_set_map_[terminal.get_name()].insert(&terminal);

    first_set_map_[Symbol::get_epsilon().get_name()].insert(&Symbol::get_epsilon()
);

    for (const Symbol& non_terminal : non_terminals_)
        first_set_map_[non_terminal.get_name()] = {};
}

```

```

bool updated = true;
while (updated)
{
    updated = false;

    for (const Production& production : productions_)
    {
        const Symbol* left_symbol = production.get_left();
        const std::vector<const Symbol*>& right_symbols =
production.get_right();

        // A -> /epsilon
        if (right_symbols.front() == &Symbol::get_epsilon())
        {
            auto [it, success] =
first_set_map_[left_symbol->get_name()].insert(&Symbol::get_epsilon());
            if (success)
                updated = true;
            continue;
        }
        // A -> X1 X2 ... Xn
        bool all_nullable = true;
        for (const Symbol* symbol : right_symbols)
        {
            // 添加 First(symbol)的非/epsilon 符号
            const std::unordered_set<const Symbol*>& first_set =
first_set_map_[symbol->get_name()];
            for (const Symbol* f : first_set)
            {
                if (f != &Symbol::get_epsilon())
                {
                    auto [it, success] =
first_set_map_[left_symbol->get_name()].insert(f);
                    if (success)
                        updated = true;
                }
            }
        }
    }
}

```

```

        // First(symbol)不含/epsilon
        auto it = first_set.find(&Symbol::get_epsilon());
        if (it == first_set.end())
        {
            all_nullable = false;
            break;
        }
    }
    // 添加/epsilon
    if (all_nullable)
    {
        auto [it, success] =
first_set_map_[left_symbol->get_name()].insert(&Symbol::get_epsilon());
        if (success)
            updated = true;
    }
}
}
}

```

其中，对于 follow 集的计算，实现方式：对于开始符号，follow 集为"\$"；对于其它非终结符，通过以下过程完成——遍历产生式，对于产生式 $A \rightarrow X_1X_2...X_n$ ，遍历每个非终结符 X_k ，计算其后面序列 $X_{k+1}, ..., X_n$ 的 first 集，将序列的 first 集中非 epsilon 符号加入当前非终结符 X_k 的 follow 集，如果序列的 first 集含有 epsilon，则将 A 的 follow 集中的符号加入当前非终结符 X_k 的 follow 集。由于 follow 集不含重复项，对于产生式的每轮遍历，都会记录是否更新了 follow 集，如果更新了 follow 集，重复遍历产生式进行下一轮计算，否则，follow 集计算完成。

```

void Grammar::compute_follow_sets()
{
    follow_set_map_.clear();

    for (const Symbol& non_terminal : non_terminals_)
        follow_set_map_[non_terminal.get_name()];
    if (start_symbol_)

follow_set_map_[start_symbol_->get_name()].insert(&Symbol::get_end());

    bool updated = true;
    while (updated)

```

```

{
    updated = false;

    for (const Production& production : productions_)
    {
        const Symbol* left_symbol = production.get_left();
        const std::vector<const Symbol*>& right_symbols =
production.get_right();

        for (std::size_t i = 0; i < right_symbols.size(); ++i)
        {
            // 遍历非终结符
            const Symbol* symbol = right_symbols[i];
            if (symbol->get_type() != Symbol::Type::NonTerminal)
                continue;

            // 计算当前符号后面序列的 First 集
            std::vector<const Symbol*>
after_symbol(right_symbols.begin() + i + 1, right_symbols.end());
            std::unordered_set<const Symbol*> first_of_sequence =
compute_first_of_sequence(after_symbol);
            // 添加非'/epsilon'符号
            for (const Symbol* f : first_of_sequence)
            {
                if (f != &Symbol::get_epsilon())
                {
                    auto[it, success] =
follow_set_map_[symbol->get_name()].insert(f);
                    if (success)
                        updated = true;
                }
            }

            // symbol 之后存在'/epsilon'
            auto it = first_of_sequence.find(&Symbol::get_epsilon());
            if (it != first_of_sequence.end())
            {

```



```

std::vector<std::string> lines;
if (read_lines(lines, file_path) == false)
    return false;

// 查找 "%%"
std::size_t index = 0;
while (index < lines.size() && lines[index].find("%%") == std::string::npos)
    ++index;

if (parse_declaration(lines, 0, index, grammar) == false)
    return false;
if (parse_rule(lines, index + 1, lines.size() - index - 1, grammar) == false)
    return false;

return true;
}

```

其中，`read_lines` 逐行读取文件，同时利用变量记录是否处于读取注释状态，遍历每行字符，如果不在读取注释状态，检查当前是否处于 `/*` 处，如果处于 `/*` 处，读取位置移至 `/*` 之后，当前处于读取注释状态，否则将当前字符记入当前行；如果处于读取注释状态，检查当前是否处于 `*/` 处，如果处于 `*/` 处，读取位置移至 `*/` 之后，当前不在读取注释状态，否则跳过当前字符。

```

bool SyntaxRuleParser::read_lines(std::vector<std::string>& lines, const
std::filesystem::path& file_path)
{
    std::ifstream file_in(file_path);
    if (file_in.is_open() == false)
        return false;

    std::string line;
    bool in_comment { false };

    while (std::getline(file_in, line))
    {
        std::size_t index = 0;
        std::string data;
        while (index < line.size())
        {
            if (in_comment)

```

```

        {
            if (index + 1 < line.size() && line[index] == '*' && line[index +
1] == '/')
            {
                in_comment = false;
                index += 2;
            }
            else
                ++index;
        }
        else
        {
            if (index + 1 < line.size() && line[index] == '/' && line[index +
1] == '*')
            {
                in_comment = true;
                index += 2;
            }
            else
            {
                data.push_back(line[index]);
                ++index;
            }
        }
    }

    if (data.empty() == false)
        lines.push_back(data);
}

file_in.close();

return true;
}

```

其中，`parse_declaration` 解析包含符号数据的行，实现方式：根据开头标识将不同符号加入文法中，对于"`%token`"标识，加入一般终结符，对于"`%left`"、"`%right`"标识，加入终结符，将其设为操作符，并且记录优先级、结合性，其中优先级根据行数确定，结合性根据"`%left`"、"`%right`"设置，对于"`%start`"标识，加入非终结符并设为开始符号。

```

bool SyntaxRuleParser::parse_declaration(const std::vector<std::string>& lines,
std::size_t begin, std::size_t size, Grammar& grammar)
{
    std::string token;
    unsigned int precedence = 1;
    for (std::size_t index = begin; index < begin + size; ++index)
    {
        std::string type;
        std::istringstream str_in(lines[index]);
        str_in >> type;

        if (str_in.fail())
        {
            str_in.clear();
            continue;
        }
        if (type == "%token")
        {
            while (str_in >> token)
                grammar.add_terminal(token);
            continue;
        }
        if (type == "%left")
        {
            while (str_in >> token)
                grammar.set_operator(token, precedence, Associativity::Left);
            ++precedence;
            continue;
        }
        if (type == "%right")
        {
            while (str_in >> token)
                grammar.set_operator(token, precedence, Associativity::Right);
            ++precedence;
            continue;
        }
        if (type == "%start")
        {

```

```

        str_in >> token;
        grammar.add_non_terminal(token);
        grammar.set_start_symbol(token);
        continue;
    }
}
return true;
}

```

其中，`parse_rule` 解析包含产生式的行，实现方式：使用变量记录当前读取状态，状态 0——读取左侧非终结符，将非终结符加入文法，遇到":"进入状态 1；状态 1：读取"："，进入状态 2；状态 2：读取右侧符号，遇到"|", 将当前产生式加入文法，遇到";", 将当前产生式加入文法并进入状态 0。

```

bool    SyntaxRuleParser::parse_rule(const    std::vector<std::string>&    lines,
std::size_t begin, std::size_t size, Grammar& grammar)
{
    std::string left_name;
    std::vector<std::string> right_names;

    unsigned int state = 0;
    for (std::size_t index = begin; index < begin + size; ++index)
    {
        std::istringstream str_in{lines[index]};
        std::string token;

        while (str_in >> token)
        {
            switch (state)
            {
            case 0:
                if (token == ":" || token == ";")
                {
                    std::cerr << "Invalid production" << '\n';
                    return false;
                }
                left_name = token;
                if (grammar.get_non_terminal(token) == nullptr)
                    grammar.add_non_terminal(token);
                state = 1;
            }
        }
    }
}

```

```

        break;
    case 1:
        if (token != ":")
        {
            std::cerr << "Invalid production" << '\n';
            return false;
        }
        state = 2;
        break;
    case 2:
        if (token == "|")
        {
            grammar.add_production(left_name, right_names);
            right_names.clear();
        }
        else if (token == ";")
        {
            grammar.add_production(left_name, right_names);
            right_names.clear();
            state = 0;
        }
        else
        {
            right_names.push_back(token);
            if (grammar.get_symbol(token) == nullptr)
                grammar.add_non_terminal(token);
        }
        break;
    }
}

return true;
}

```

6、LR1 项

LR1 项记录产生式、前瞻符号以及输入位置。

```

class LRItem
{

```

```

public:
    bool operator==(const LRItem& other) const;

private:
    const Production* production_;
    uint32_t dot_pos_;
    const Symbol* lookahead_;
};

```

7、LR1 状态

LR1 状态记录状态 id，以及 1 个 LR1 项的集合。

```

class LRState
{
public:
    bool operator==(const LRState& other) const;

private:
    uint32_t id_;
    std::unordered_set<const LRItem*> items_;
};

```

8、LR1 分析

LR1 分析器记录 1 个拓广文法开始符号、1 个拓广文法新增产生式、LR1 项数据、LR1 状态、action 表、goto 表以及 1 个抽象语法树，其中 LR1 项数据保证唯一性，LR1 状态以及其它类只能获取其指针。

LR1 分析器提供状态图的构建、action 表和 goto 表的构建，其中的语法分析接口 parse 仅用于调试。另外，LR1 还提供 LR1 到 LALR1 的合并函数 merge_states，但 merge_states 仅将同心集合并到同一状态，并未处理 LR1 原状态与 LALR1 新状态的映射关系，merge_states 还会修改 LR1 状态数据，因此不能用于 LR1 分析。

```

class LRParser
{
public:
    bool build_states();
    bool construct_tables();
    void merge_states();
    bool parse(const std::vector<Token>& token_strings);

private:
    LRState get_closure(const LRState& state);
    LRState get_next_state(const LRState& state, const Symbol* next_symbol);
};

```

```

    int32_t find_state_by_items(const std::unordered_set<const LRItem*> & items)
const;
    Action resolve_action_conflict(const Action& existing, const Action& incoming,
const Symbol* terminal) const;
    const Symbol* get_production_precedence_symbol(const Production*
production) const;

private:
    const Grammar* grammar_;
    Symbol start_symbol_;
    Production start_production_;
    std::unordered_set<LRItem> items_;
    uint32_t current_state_id_;
    std::vector<LRState> states_;
    std::unordered_map<uint32_t, std::unordered_map<const Symbol*, Action>>
action_table_;
    std::unordered_map<uint32_t, std::unordered_map<const Symbol*, uint32_t>>
goto_table_;
    ASTTree ast_tree_;
    std::vector<const ASTNode*> ast_nodes_stack_;
};

```

其中，build_states 基于原始文法构造 LR1 状态图，实现方式：获取原始文法的开始符号，创建拓广文法的额外产生式。

其中，拓广文法的开始符号在 LR1 解析器构造时创建，新增的符号、产生式不会加入原始文法。通过以下过程构建 LR1 状态，同时更新 action 表、goto 表——遍历当前状态集合，对于当前状态的每个 LR1 项，如果 LR1 项的输入位置未到结尾，将其下个符号记录，这样得到当前状态的所有可能输入符号。对于每个输入符号，计算当前状态的下个状态，并记录下个状态的 id，如果计算结果状态已经存在，记录新的状态，避免状态重复。根据当前状态 id、下个状态 id，以及当前输入符号，更新 action 表或者 goto 表，如果输入符号为终结符，添加移进动作；如果当前输入符号为非终结符，添加 goto 表项。其中，更新 action 表时，如果已经存在动作，需要处理冲突。

```

bool LRParser::build_states()
{
    items_.clear();
    states_.clear();
    action_table_.clear();
    goto_table_.clear();
}

```



```

std::cout << "Starting build_states" << "\n";

const Symbol* original_start_symbol = grammar_->get_start_symbol();
if (original_start_symbol == nullptr)
    return false;

uint32_t start_production_id = grammar_->get_current_production_id();
start_production_.reset(start_production_id, &start_symbol_,
std::vector<const Symbol*>{original_start_symbol});
LRItem new_item{&start_production_, 0, &Symbol::get_end()};
auto [it, success] = items_.insert(new_item);
const LRItem* start_item = &(*it);

LRState start_state = get_closure(LRState{current_state_id_,
std::unordered_set<const LRItem*>{start_item}});
states_.push_back(start_state);
++current_state_id_;

for (std::size_t i = 0; i < states_.size(); ++i)
{
    const LRState current_state = states_[i];
    // 记录可能输入符号
    const auto& items = current_state.get_items();
    std::unordered_set<const Symbol*> symbols;
    for (const LRItem* item : items)
    {
        if (item->is_dot_at_end() == false)
            symbols.insert(item->get_next_symbol());
    }

    for (const Symbol* symbol : symbols)
    {
        LRState next_state = get_next_state(current_state, symbol);
        // 添加状态
        if (next_state.get_items().empty())
            continue;
        // 检查是否已存在
        uint32_t target_state_id = 0;

```

```

        auto it = std::find_if(
            states_.begin(), states_.end(),
            [&next_state](const LRState& s) { return s.get_items() ==
next_state.get_items(); }
        );
        if (it == states_.end())
        {
            states_.push_back(next_state);
            target_state_id = current_state_id_;
            ++current_state_id_;
        }
        else
        {
            target_state_id = it->get_id();
        }
        // 更新 action, goto
        uint32_t source_state_id = current_state.get_id();
        if (symbol->get_type() == Symbol::Type::Terminal)
        {
            Action action{Action::Type::Shift, target_state_id};
            auto& row = action_table_[source_state_id];
            auto act_it = row.find(symbol);
            if (act_it == row.end())
                row[symbol] = action;
            else
                act_it->second = resolve_action_conflict(act_it->second,
action, symbol);
        }
        else if (symbol->get_type() == Symbol::Type::NonTerminal)
        {
            goto_table_[source_state_id][symbol] = target_state_id;
        }
    }
}

std::cout << "Total states created: " << states_.size() << "\n";
return true;
}

```

其中, construct_tables 对 build_states 之后的 action 表进行更新, 将规约、接受动作加入 action 表。实现方式: 遍历所有 LR1 状态, 对于每个状态遍历 LR1 项, 检查当前 LR1 项的输入位置是否到达末尾, 如果到达末尾, 根据产生式的左侧非终结符是否为拓广文法开始符号, 创建规约或者接受动作, 添加动作到 action 表, 如果 action 表中已经存在动作, 需要处理冲突。

```
bool LRParser::construct_tables()
{
    if (states_.empty())
        return true;

    std::cout << "Starting construct_tables" << '\n';

    for (const LRState& state : states_)
    {
        uint32_t state_id = state.get_id();

        for (const LRItem* item : state.get_items())
        {
            const Production* production = item->get_production();
            uint32_t dot_pos = item->get_dot_pos();
            const Symbol* lookahead = item->get_lookahead();

            // 可规约项
            if (dot_pos == production->get_right().size())
            {
                Action action;
                if (production->get_left() == &start_symbol_)
                {
                    action = Action{Action::Type::Accept, 0};
                }
                else
                {
                    action = Action{Action::Type::Reduce,
production->get_id()};
                }

                auto& row = action_table_[state_id];
                auto it = row.find(lookahead);
```

```

        if (it == row.end())
            row[lookahead] = action;
        else
            it->second = resolve_action_conflict(it->second, action,
lookahead);
        continue;
    }
}

std::cout << "Construct_tables completed\n";
return true;
}

```

在构建 LR1 状态时，需要获取下个 LR1 状态，通过 `get_next_state` 实现，实现方式：遍历当前状态的所有 LR1 项，对于其中前瞻符号是输入符号的 LR1 项，移动输入符号的位置，得到新的 LR1 项，记录所有新的 LR1 项。如果没有新的 LR1 项，返回 1 个无效 LR1 状态，否则，对所有新的 LR1 项组成的集合进行闭包计算，得到新的 LR1 状态。

```

LRState LRParser::get_next_state(const LRState& state, const Symbol*
next_symbol)
{
    std::unordered_set<const LRItem*> next_items;
    std::unordered_set<const LRItem*> items = state.get_items();
    for (const LRItem* item : items)
    {
        if (item->is_dot_at_end() == false && item->get_next_symbol() ==
next_symbol)
        {
            LRItem new_item{item->get_production(), item->get_dot_pos() +
1, item->get_lookahead()};
            auto [it, success] = items_.insert(new_item);
            next_items.insert(&(*it));
        }
    }

    if (next_items.empty())
        return LRState{current_state_id_, {}};

    return get_closure(LRState{current_state_id_, next_items});
}

```

```
}
```

闭包计算实现方式：执行如下过程——遍历 LR1 项，对于下个符号是非终结符的 LR1 项，获取下个非终结符之后序列的 first 集，作为新的前瞻符号集合，如果 first 集中含有 epsilon，则把前瞻符号集合中的 epsilon 改为当前 LR1 项的前瞻符号。之后，对于每个新的前瞻符号，找到对应的产生式，构造新的 LR1 项，加入当前状态。使用变量记录当前状态是否添加新的 LR1 项，如果仍有更新，继续下一轮的遍历，否则返回当前状态。

```
LRState LRParser::get_closure(const LRState& state)
{
    LRState result = state;

    bool updated = true;
    while (updated)
    {
        updated = false;

        const std::unordered_set<const LRItem*>
items{result.get_items().begin(), result.get_items().end()};
        for (const LRItem* item : items)
        {
            if (item->is_dot_at_end())
                continue;

            const Symbol* next_symbol = item->get_next_symbol();
            if (next_symbol->get_type() != Symbol::Type::NonTerminal)
                continue;

            // 计算 lookahead
            // beta
            std::vector<const Symbol*> beta;
            const std::vector<const Symbol*>& right_symbols =
item->get_production()->get_right();
            for (std::size_t i = item->get_dot_pos() + 1; i < right_symbols.size();
++i)
                beta.push_back(right_symbols[i]);
            std::unordered_set<const Symbol*> lookaheads =
grammar_->compute_first_of_sequence(beta);
            // 当前 lookahead
            auto epsilon_it = lookaheads.find(&Symbol::get_epsilon());
```

```

        if (epsilon_it != lookaheads.end())
        {
            lookaheads.erase(&Symbol::get_epsilon());
            lookaheads.insert(item->get_lookahead());
        }

        const auto& productions = grammar_->get_productions();
        for (const Production& production : productions)
        {
            if (production.get_left() != next_symbol)
                continue;

            for (const Symbol* lookahead : lookaheads)
            {
                LRItem new_item{&production, 0, lookahead};
                auto [it, success] = items_.insert(new_item);

                const LRItem* i = result.add_item(&(*it));
                if (i)
                    updated = true;
            }
        }
    }

    return result;
}

```

其中，解决冲突通过 `resolve_action_conflict` 实现，实现方式：如果动作相同，无须处理，如果原有动作与新的动作中有 1 个是错误动作，那么使用非错误动作。如果不是上述情况，依次按照移进-规约冲突、规约-规约冲突进行处理，对于移进-规约冲突，先根据优先级判断，返回优先级较高的一方的动作，对于移进项目，如果两方优先级相同，如果移进项目左结合，返回规约动作，否则返回移进动作；对于规约-规约冲突，返回优先级较高的一方的动作。

```

Action LRParser::resolve_action_conflict(const Action& existing, const Action&
incoming, const Symbol* terminal) const
{
    if (existing.get_type() == incoming.get_type())
        return existing;
}

```

```

if (existing.get_type() == Action::Type::Error)
    return incoming;
if (incoming.get_type() == Action::Type::Error)
    return existing;

if (existing.get_type() == Action::Type::Accept || incoming.get_type() ==
Action::Type::Accept)
    return existing.get_type() == Action::Type::Accept ? existing : incoming;

const Action* shift_action = nullptr;
const Action* reduce_action = nullptr;
if (existing.get_type() == Action::Type::Shift)
    shift_action = &existing;
else if (existing.get_type() == Action::Type::Reduce)
    reduce_action = &existing;

if (incoming.get_type() == Action::Type::Shift)
    shift_action = &incoming;
else if (incoming.get_type() == Action::Type::Reduce)
    reduce_action = &incoming;

if (shift_action && reduce_action)
{
    uint32_t shift_prec = terminal->get_precedence();
    const      Production*      reduce_prod      =
grammar->get_production(reduce_action->get_arg());
    const      Symbol*          reduce_symbol     =
get_production_precedence_symbol(reduce_prod);
    uint32_t    reduce_prec      =      reduce_symbol      ?
reduce_symbol->get_precedence() : 0;
    Associativity assoc = terminal->get_associativity();

    if (shift_prec > reduce_prec)
        return *shift_action;
    if (shift_prec < reduce_prec)
        return *reduce_action;
}

```

```

        if (assoc == Associativity::Left)
            return *reduce_action;
        if (assoc == Associativity::Right)
            return *shift_action;

        return existing;
    }

    if (existing.get_type() == Action::Type::Reduce && incoming.get_type() ==
        Action::Type::Reduce)
    {
        const      Production*      prod_existing      =
grammar_ -> get_production(existing.get_arg());
        const      Production*      prod_incoming      =
grammar_ -> get_production(incoming.get_arg());
        const      Symbol*          symbol_existing     =
get_production_precedence_symbol(prod_existing);
        const      Symbol*          symbol_incoming     =
get_production_precedence_symbol(prod_incoming);
        uint32_t    prec_existing    =      symbol_existing    ?
symbol_existing->get_precedence() : 0;
        uint32_t    prec_incoming    =      symbol_incoming    ?
symbol_incoming->get_precedence() : 0;

        if (prec_existing > prec_incoming)
            return existing;
        if (prec_incoming > prec_existing)
            return incoming;
        return existing;
    }

    return existing;
}

```

9、语法分析源码生成

语法分析源码从 LR1 解析器获取数据，实现语法分析接口。

在语法分析源码中，SyntaxParser 使用简化后的数据结构记录了文法、action 表、goto 表，以便与其它编译过程交接，利用这些记录，SyntaxParser 提供语法解析接口 parse，接收 token 流，构建抽象语法树。


```

        // generate .h file
        h << R"(
#pragma once
#include "token.h"
#include <stdint>
#include <string>
#include <filesystem>
#include <unordered_map>
#include <vector>
);

        h << R"(
struct Production
{
    std::string left;
    std::vector<std::string> right;
};

struct TableEntry
{
    uint32_t state;
    std::string symbol;
    std::string type;
    uint32_t next_state;
};

struct ASTNode
{
    std::string type;
    std::string value;
    std::vector<ASTNode*> children;
};

);

        h << R"(
class SyntaxParser
{
public:
    SyntaxParser();
    ~SyntaxParser();

```

```

bool parse(const std::vector<Token>& tokens);

std::vector<Production> get_grammar();
std::vector<TableEntry> get_action_table();
std::vector<TableEntry> get_goto_table();
ASTNode* get_ast_tree();
private:
    void clear_ast_tree();
    const TableEntry* find_action(uint32_t state, const std::string& symbol)
const;
    const TableEntry* find_goto(uint32_t state, const std::string& symbol) const;

private:
    std::vector<Production> grammar;
    std::vector<TableEntry> action_table;
    std::vector<TableEntry> goto_table;
    std::unordered_map<uint32_t, std::unordered_map<std::string, std::size_t>>
action_index;
    std::unordered_map<uint32_t, std::unordered_map<std::string, std::size_t>>
goto_index;
    ASTNode* ast_tree;
};
)";

```

其中，parse 函数完成最终语法分析，实现方式如下：使用栈记录当前状态、AST 树节点。状态栈中 push 状态 0，执行如下过程——获取当前 token，在 action 表中查找对应动作，对于移进动作，状态栈 push 下个状态，AST 节点栈 push 新的 AST 节点；对于规约动作，获取规约的产生式，根据产生式右部的长度，将 AST 树节点栈中 pop 对应数量的 AST 树节点，同时状态栈 pop 对应数量的状态，创建新的 AST 树节点作为父节点，之前指定数量的 AST 树节点作为子节点，顺序与出栈顺序相反。AST 节点站 push 父节点，在 goto 表中查找规约后的下个状态，状态栈 push 下个状态；对于接受动作，记录 AST 节点栈的顶部为 AST 树的根节点，分析完成；对于错误动作，返回错误。

```

cpp << R"(
bool SyntaxParser::parse(const std::vector<Token>& tokens)
{
    std::stack<uint32_t> state_stack;
    std::stack<ASTNode*> node_stack;

```

```

state_stack.push(0);
size_t pos = 0;

clear_ast_tree();

while (true)
{
    uint32_t cur_state = state_stack.top();
    std::string cur_sym;

    if (pos < tokens.size())
        cur_sym = tokens[pos].type;
    else
        cur_sym = "$";

    const TableEntry* action = find_action(cur_state, cur_sym);

    if (!action)
    {
        std::cerr << "Parse error: state " << cur_state
                    << " symbol " << cur_sym << "\n";
        return false;
    }

    // ===== Shift
    if (action->type == "Shift")
    {
        state_stack.push(action->next_state);

        ASTNode* n = new ASTNode;
        n->type = cur_sym;
        n->value = tokens[pos].value;
        node_stack.push(n);

        pos++;
    }
}

```

```

// ===== Reduce
else if (action->type == "Reduce")
{
    uint32_t rule_idx = action->next_state;
    const Production& rule = grammar[rule_idx];

    std::vector<ASTNode*> children;
    for (size_t i = 0; i < rule.right.size(); i++)
    {
        children.push_back(node_stack.top());
        node_stack.pop();
        state_stack.pop();
    }

    std::reverse(children.begin(), children.end());

    ASTNode* parent = new ASTNode;
    parent->type = rule.left;
    parent->value = "";
    parent->children = children;

    node_stack.push(parent);

    uint32_t top_state = state_stack.top();
    const TableEntry* goto_entry = find_goto(top_state, rule.left);
    if (!goto_entry)
        return false;

    state_stack.push(goto_entry->next_state);
}

// ===== Accept
else if (action->type == "Accept")
{
    ast_tree = node_stack.top();
    return true;
}

```

```
        else
        {
            return false;
        }
    }
}

);
```